

Design & Analysis of Software Systems

Mid-Semester Exam – Complete Revision Notes

Covers: Spring 2024 & Spring 2025 Papers · All Topics & Worked Examples

Contents

1	Software Process / Lifecycle Models	3
1.1	Common Process Models	3
1.2	Lifecycle Model Selection (2025 Q18)	3
2	Project Planning & Scheduling	3
2.1	Key Definitions	3
2.2	Critical Path Method (CPM) – Theory	4
2.2.1	CPM Formulae	4
2.2.2	Worked Example – 2024 Q9 (6 Tasks, 16-week target)	4
2.2.3	Worked Example – 2025 Q19 (9 Tasks)	5
3	Requirements Engineering	5
3.1	Activities of Requirements Engineering (Both Years Q5)	6
3.2	Characteristics of Good Requirements (2024 Q8, 2025 Q17)	6
3.3	Requirements Elicitation Techniques (2025 Q8)	7
4	Agile / Scrum Concepts	7
4.1	Product Backlog (2024 Q3, 2025 Q14)	7
4.2	TDD – Test-Driven Development (2025 Q1)	7
4.3	Agile Progress Tracking (2025 Q6)	7
5	UML Diagrams	7
5.1	UML Class Diagram Relationships (2024 Q5)	7
5.1.1	Quick UML Cheat Sheet	8
5.2	Multiplicities in UML (2025 Q9)	8
6	Use Case Diagrams & Scenarios	8
6.1	Use Case Actors (2024 Q1a)	8
6.2	Identifying Use Cases (2025 Q7)	8
6.3	Use Case Diagram – Course Registration System (2024 Q10)	9
6.4	Use Case Diagram – Space Station Parking Garage (2025 Q20)	9
7	Testing	10
7.1	Regression Testing (2024 Q4)	10
7.2	Acceptance Testing (2024 Q1f)	10
7.3	Assertions in Unit Testing (2025 Q10)	11
7.4	Equivalence Class Partitioning (2024 Q6)	11
7.5	Edge Cases – calculateRepayment (2024 Q11, 2025 Q15)	11
7.6	Unit Tests – InventoryManager (2024 Q12)	12
8	Verification & Validation (V & V)	13
9	Quality Management – Deming’s Cycle	13
10	Concurrent Development & Git	13
11	Project Tracking – Megathon Example	14

12 Quick-Reference Answer Sheet	14
12.1 Multiple Choice – Definitive Answers	14
12.2 One-Liners to Memorise	14
13 CPM Quick-Reference Formula Sheet	15

Software Process / Lifecycle Models

Common Process Models

Key Models – Know These

Waterfall: Sequential, phase-by-phase. Each phase finishes before the next begins. Simple but inflexible; requirements must be known upfront.

Spiral (Risk-Driven): Iterative, risk-driven model (the answer the exam wants). Each loop: Plan → Risk Analysis → Develop/Test → Evaluate. Best when risks are high or requirements are unclear.

Prototype: Build a throwaway prototype to clarify requirements, then build the real system. Good for UI-heavy or unclear-requirements projects.

Scrum / Agile: Iterative sprints (2–4 weeks), continuous delivery, daily stand-ups, backlogs. Best for teams where requirements change frequently.

Incremental: Deliver working software in increments. Core features first, then add.

Exam Fact

“The Spiral process model is a risk-driven process model.” (appeared in both 2024 & 2025)

Lifecycle Model Selection (2025 Q18)

Q: Team of 4, 6 months, mobile UI-evaluation app – which lifecycle?

Recommended: Agile / Scrum

Rationale:

- Small team (4 people) – Scrum roles (PO, SM, devs) map cleanly.
- Mobile UI evaluation requires iterative user feedback – cannot fully specify requirements upfront.
- 6 months allows 8–12 two-week sprints; features can be prioritised per sprint.
- Continuous testing ensures the evaluation logic is correct across many apps.
- Agile’s inspect-and-adapt cycle handles changing evaluation criteria gracefully.

Why not Waterfall? UI work is hard to fully spec upfront; changes are expected.

Why not Spiral? Spiral suits large, high-risk projects – overhead is disproportionate here.

Project Planning & Scheduling

Key Definitions

Fill-in-the-blank – Both Years

- **Schedule:** The timetable that specifies when each activity should *start and finish*. (Answer: schedule)
- **Request for Proposal (RFP):** Document written by the customer specifying needs and requirements, inviting vendors to propose solutions. (Answer: **b. request for proposal**)
- **CPM = Critical Path Method** – identifies the longest dependency chain in a project.

Critical Path Method (CPM) – Theory

CPM Primary Benefit (2025 Q4)

CPM identifies the longest sequence of dependent tasks (the critical path) to determine the minimum project duration, and shows which tasks cannot be delayed without delaying the project. It facilitates effective scheduling by highlighting where to focus management attention.

2.2.1 CPM Formulae

$$ES(j) = \max_{i \rightarrow j}(EF(i)), \quad EF(j) = ES(j) + \text{Duration}(j)$$

$$LF(j) = \min_{j \rightarrow k}(LS(k)), \quad LS(j) = LF(j) - \text{Duration}(j)$$

$$\text{Slack}(j) = LS(j) - ES(j) = LF(j) - EF(j)$$

Tasks with **Slack = 0** are on the **critical path**.

2.2.2 Worked Example – 2024 Q9 (6 Tasks, 16-week target)

Task	Est. Time	Act. Finish	Predecessor
A	3	4	–
B	3	6	–
C	4	–	A
D	5	–	A
E	6	–	B
F	7	–	C, D, E

Forward Pass (using estimated durations):

Task	ES	Duration	EF
A	0	3	3
B	0	3	3
C	3 (after A)	4	7
D	3 (after A)	5	8
E	3 (after B)	6	9
F	$\max(7, 8, 9) = 9$	7	16

Backward Pass (LF of project = 16):

Task	LF	Duration	LS	Slack
F	16	7	9	0 (Critical)
E	9	6	3	0 (Critical)
D	9	5	4	1
C	9	4	5	2
B	3	3	0	0 (Critical)
A	$\min(LS_C, LS_D) = \min(5, 4) = 4$	3	1	1

Summary – 2024 Q9

Critical Path: B → E → F (total = 3+6+7 = 16 weeks – exactly on time with estimates).

Can it finish on time? Based on *actual* finish times: B actually took 6 weeks (not 3). This pushes E's ES to 6, making EF(E)=12, and EF(F)=19. **Project is now 3 weeks late.**

Tasks to focus on: Crash tasks on the critical path – specifically **E and F** – by adding resources

or overlapping work.

2.2.3 Worked Example – 2025 Q19 (9 Tasks)

Task	Time (hrs)	Predecessor
A	15	–
B	20	A
C	40	A
D	20	B, C
E	10	D
F	10	E
G	25	F
H	20	F
I	15	G, H

Forward Pass:

Task	ES	Duration	EF
A	0	15	15
B	15	20	35
C	15	40	55
D	$\max(35, 55) = 55$	20	75
E	75	10	85
F	85	10	95
G	95	25	120
H	95	20	115
I	$\max(120, 115) = 120$	15	135

Backward Pass (LF = 135):

Task	LF	LS	Slack	
I	135	120	0	Critical
G	120	95	0	Critical
H	120	100	5	
F	$\min(95, 100) = 95$	85	0	Critical
E	85	75	0	Critical
D	75	55	0	Critical
C	55	15	0	Critical
B	55	35	20	
A	$\min(35, 15) = 15$	0	0	Critical

Summary – 2025 Q19

Critical Path: A → C → D → E → F → G → I

Total project time: 135 man-hours

Task H has 5 hrs of slack; **Task B** has 20 hrs of slack.

Activities of Requirements Engineering (Both Years Q5)

Answer: B**Elicitation → Modeling → Domain Analysis → Specification**

- **Elicitation:** Gather requirements from stakeholders (interviews, workshops, observation).
- **Modeling:** Represent requirements as diagrams/models (use cases, data flow, etc.).
- **Domain Analysis:** Understand the problem domain and existing solutions.
- **Specification:** Write the Software Requirements Specification (SRS) document.

Wrong options to avoid: *Code Inspections* and *Design Reviews* are QA activities, not requirements activities.

Characteristics of Good Requirements (2024 Q8, 2025 Q17)

SMART + additional properties:**Good Requirements are:**

1. **Correct** – accurately describes what the system must do
2. **Unambiguous** – only one interpretation possible
3. **Complete** – no missing conditions or cases
4. **Consistent** – no conflicts with other requirements
5. **Verifiable / Testable** – can be confirmed by test/inspection
6. **Traceable** – linked to its origin
7. **Feasible** – achievable within constraints
8. **Prioritised** – importance/urgency indicated

Bad Requirement Analysis – “The cash register program must show the correct prices for all items bought by the customer and print the total at the bottom.”**Problems (what it lacks):**

- **Not Verifiable/Testable:** “Correct prices” is undefined – correct according to what source?
- **Ambiguous:** “print the total at the bottom” – bottom of screen? Bottom of receipt? Font size?
- **Incomplete:** Does not specify currency, formatting, tax handling, discounts, or error cases.
- **No measurable acceptance criteria.**

Rewritten (good requirements):

- The cash register system shall display the unit price for each scanned item, retrieved from the central product database, within 0.5 seconds of scanning.
- The system shall display the itemised receipt showing each item name, quantity, and unit price.
- The system shall calculate and display the subtotal, applicable tax (at the current configured tax rate), and grand total at the bottom of the receipt.
- The grand total shall match the sum of all item prices plus tax, accurate to two decimal places.
- If a scanned item is not found in the database, the system shall display an error message “Item not found” and prompt the cashier to enter the price manually.

Requirements Elicitation Techniques (2025 Q8)

Best technique for high-quality requirements elicitation? Answer: B

Joint Application Development (JAD) sessions – Facilitated workshops where developers, stakeholders, and users collaborate to define requirements together. Produces shared understanding, immediate conflict resolution, and richer requirements than surveys or emails alone.

Agile / Scrum Concepts

Product Backlog (2024 Q3, 2025 Q14)

What does product backlog mean?

The **product backlog** is an ordered (prioritised) list of all features, functions, requirements, enhancements, and fixes that constitute the changes to be made to a product in future releases.

- Owned and managed by the **Product Owner**.
- Items are called **user stories** or backlog items.
- At the start of each sprint, the team pulls the highest-priority items into the **sprint backlog**.
- It is a **living document** – new items are added and priorities change continuously.
- Each item should have an estimate (story points), a description, and acceptance criteria.

TDD – Test-Driven Development (2025 Q1)

Primary purpose of TDD? Answer: B

TDD means **writing automated tests *before* writing the code**. The cycle is **Red** → **Green** → **Refactor**:

1. **Red**: Write a failing test for the desired behaviour.
2. **Green**: Write the minimum code to make the test pass.
3. **Refactor**: Clean up code without breaking tests.

Benefits: promotes better design, catches defects early, ensures every line of code is tested, acts as living documentation.

Agile Progress Tracking (2025 Q6)

Best tool for tracking progress in Agile? Answer: B

Kanban boards – visualise tasks as cards moving through columns (e.g., To Do → In Progress → Done). Provides real-time visibility, limits work-in-progress (WIP), and supports continuous flow. Burndown charts and velocity charts are also common Agile tracking tools.

UML Diagrams

UML Class Diagram Relationships (2024 Q5)

Three Relationships to Draw

(a) Generalization (Inheritance) – Phone and iPhone:

A hollow triangle arrowhead points *from subclass to superclass*.

iPhone --|> Phone (iPhone inherits from Phone)

(b) Multiplicity 1 to many – Manager and Assistant:

A plain association line with 1 on the Manager end and 1..* (or *) on the Assistant end.

Manager --1 ----- 1..* -- Assistant

(c) Strong Aggregation (Composition) – Team and Member, only Team knows about Member:

A **filled diamond** on the Team end, arrow pointing to Member (showing Team → Member navigability, unidirectional).

Team --◆→-- Member

Filled diamond = composition (strong aggregation); the Member cannot exist without the Team.

Navigability arrow on the Member end indicates only Team knows about Member (not vice versa).

5.1.1 Quick UML Cheat Sheet

Relationship	Symbol	Meaning
Association	plain line	General relationship
Dependency	dashed arrow	Uses/depends on
Generalization	hollow triangle arrow	Inheritance
Realization	dashed hollow triangle	Implements interface
Aggregation	hollow diamond	Has-a (weak)
Composition	filled diamond	Has-a (strong, lifecycle)

Multiplicities in UML (2025 Q9)

How do multiplicities enhance understanding? Answer: D

Multiplicities **specify the exact number of instances from each class that can participate in a relationship**, indicating constraints on associations (e.g., one manager can have 1..* assistants; each assistant has exactly 1 manager).

Notation	Meaning
1	Exactly one
0..1	Zero or one (optional)
* or 0..*	Zero or more
1..*	One or more
2..5	Between 2 and 5

Use Case Diagrams & Scenarios

Use Case Actors (2024 Q1a)

“Use-case actors are always people, never system devices.” – True or False?

False. Actors represent anything that interacts with the system *from outside*: people, external systems, hardware devices, timers, etc. A payment gateway, an external API, or a card reader can all be actors.

Identifying Use Cases (2025 Q7)

Most effective approach to ensure all stakeholder perspectives captured? Answer: A

Conducting interviews with key stakeholders – direct engagement ensures every user group’s perspective is heard. Combined with workshops (JAD), this is the most thorough approach.

Use Case Diagram – Course Registration System (2024 Q10)**Actors:** Student, Registrar, Department Secretary, Instructor**Use Cases:**

- Student: View course listings, Add class, Drop class, View schedule
- Registrar: View enrollment statistics, View under-enrolled classes
- Department Secretary: Add/modify/delete course listings, Schedule course section offerings
- Instructor: View registration info for taught courses
- System (implicit): Authenticate users, Store data in central DB

Main Flow – “Register for a Class” Use Case**Use Case:** Register for a Class**Actor:** Student**Precondition:** Student is authenticated in the system.**Main Flow:**

1. Student selects “View Course Listings.”
2. System displays available courses for the upcoming term.
3. Student selects a desired course section.
4. System checks: (a) seat availability, (b) no schedule conflicts, (c) student meets prerequisites.
5. System adds the course to the student’s schedule.
6. System updates the enrollment count in the central database.
7. System displays confirmation with updated schedule.

Postcondition: Student is enrolled; enrollment count incremented.**Alternate Flow 1 – No seats available (at step 4a):**

- 4a.1 System detects the section is full.
- 4a.2 System notifies the student: “Class is full.”
- 4a.3 System offers to add student to waitlist.
- 4a.4 If student accepts, system adds them to waitlist. Use case ends.

Alternate Flow 2 – Schedule conflict (at step 4b):

- 4b.1 System detects a time conflict with an already-registered course.
- 4b.2 System displays the conflicting courses.
- 4b.3 Student may choose to drop the conflicting course and retry, or cancel. Use case ends.

Use Case Diagram – Space Station Parking Garage (2025 Q20)**System:** Space Station Parking Garage Management System**Actors:** Driver (with permit), Driver (with ticket), Attendant, Operator, Permit Holder (for billing), System (timer/sensor)**Use Cases:**

- Enter Garage (via permit swipe or ticket)
- Enter Air Lock
- Exit Garage (via air lock)
- Pay Parking Fee (ticket holder – pays attendant)
- Deduct Permit Fee (permit holder – auto-charged on exit)
- View Garage Status (Operator – # parked, in air lock, waiting)
- Display Full/Available Signs (system-driven)
- Generate Usage Report (Operator – peak hours, permit usage)
- Track Vehicle Count (automated)

Main Flow – “Enter Garage with Ticket”

Use Case: Enter Garage with Ticket

Actor: Driver (ticket)

Precondition: Garage is not full; at least one air lock has space.

Main Flow:

1. Driver arrives at entrance and presses the ticket button.
2. System activates the mechanical arm to dispense the ticket.
3. Ticket is printed with current date/time stamp (entry time).
4. Driver takes the ticket via the vehicle’s automatic arm (window stays closed).
5. System checks: air lock availability.
6. System signals air lock to open outer door.
7. Driver enters the air lock; system records vehicle in air lock.
8. Outer door closes; inner door opens; driver enters the garage.
9. System increments parked vehicle count; updates status signs.
10. If garage is now full, system sets “FULL” sign.

Postcondition: Vehicle parked; count updated; ticket held by driver for payment on exit.

Alternate Flow – Air lock is full (at step 5):

- 5a.1 System detects no available air lock space.
- 5a.2 System displays “Air Lock Full – Please Wait” sign.
- 5a.3 System adds vehicle to waiting queue.
- 5a.4 When an air lock space becomes available, system signals the next waiting vehicle. Return to step 6.

Testing

Regression Testing (2024 Q4)

What is regression testing and why is it done?

Regression testing is the re-execution of a previously passing set of tests after a change (bug fix, new feature, refactoring) to verify that:

1. The change did not break any previously working functionality.
2. No new defects were introduced.

Why: Software changes are interdependent – a fix in module A can break module B. Regression testing provides a safety net, giving confidence that the system still behaves correctly after every change. It is typically automated (e.g., CI/CD pipelines run the regression suite on every commit).

Acceptance Testing (2024 Q1f)

Who conducts acceptance tests? Answer: A

End users conduct acceptance testing (also called User Acceptance Testing / UAT). They verify the system meets their real-world needs before signing off on delivery. Developers do unit/integration testing; a test team does system testing.

Assertions in Unit Testing (2025 Q10)

Fill in the blank: common assertion method that checks a condition holds true

`assertTrue()` – checks that a specific condition is `True`.

Other common assertions: `assertEqual(a, b)`, `assertNotEqual`, `assertIsNone`, `assertRaises`, `assertIn`.

Equivalence Class Partitioning (2024 Q6)

Specification: “The user shall respond yes or no.”

Three Equivalence Classes

Class	Description	Valid/Invalid
Class 1	Input is exactly "yes" (case-sensitive or insensitive)	Valid
Class 2	Input is exactly "no"	Valid
Class 3	Any other string (e.g., "maybe", "y", "", "123")	Invalid

Test cases:

- TC1: Input = "yes" → Expected: accepted (valid)
- TC2: Input = "no" → Expected: accepted (valid)
- TC3: Input = "maybe" → Expected: error/prompt again (invalid)

Edge Cases – calculateRepayment (2024 Q11, 2025 Q15)

Listing 1: Loan Repayment Function

```
function calculateRepayment(principal, annualInterestRate, years) {
    monthlyInterestRate = annualInterestRate / 12
    numberOfPayments = years * 12
    return (principal * monthlyInterestRate) /
        (1 - (1 + monthlyInterestRate)^(-numberOfPayments))
}
```

Edge Cases & Test Cases

- Zero interest rate (annualInterestRate = 0):**
Problem: Division by zero – $\text{monthlyInterestRate}=0$, denominator = $1 - (1 + 0)^{-n} = 0$.
Test: principal=100000, rate=0, years=10. Expected: simple division ($100000 / 120 = \$833.33$) or handled error.
- Zero or negative principal:**
Problem: Nonsensical – you cannot have a loan of \$0 or negative amount.
Test: principal=0, rate=5%, years=20. Expected: error / returns 0.
Test: principal=-50000. Expected: error raised.
- Zero or negative loan term (years = 0):**
Problem: $\text{numberOfPayments} = 0$, denominator = $1 - (1 + r)^0 = 0$, division by zero.
Test: principal=100000, rate=5%, years=0. Expected: error.
- Very large interest rate (> 1 i.e. 100%):**
Problem: Mathematically valid but practically nonsensical (e.g., 1000% interest).
Test: principal=1000, rate=10.0 (1000%), years=1. Expected: validate rate is in [0,1] range.
- Negative interest rate:**
Problem: Negative rates are unusual; formula may produce unexpected results.
Test: principal=100000, rate=-0.02, years=5. Expected: error or specific handling.

Unit Tests – InventoryManager (2024 Q12)

Listing 2: InventoryManager Unit Tests

```
import unittest

class TestInventoryManager(unittest.TestCase):

    def setUp(self):
        self.inv = InventoryManager()

    # --- add_product ---
    def test_add_new_product(self):
        self.inv.add_product("apple", 10)
        self.assertEqual(self.inv.get_product_quantity("apple"), 10)

    def test_add_increases_existing_quantity(self):
        self.inv.add_product("apple", 10)
        self.inv.add_product("apple", 5)
        self.assertEqual(self.inv.get_product_quantity("apple"), 15)

    def test_add_zero_quantity_raises_error(self):
        with self.assertRaises(ValueError):
            self.inv.add_product("apple", 0)

    def test_add_negative_quantity_raises_error(self):
        with self.assertRaises(ValueError):
            self.inv.add_product("apple", -5)

    # --- remove_product ---
    def test_remove_product_reduces_quantity(self):
        self.inv.add_product("apple", 10)
        self.inv.remove_product("apple", 3)
        self.assertEqual(self.inv.get_product_quantity("apple"), 7)

    def test_remove_all_deletes_product(self):
        self.inv.add_product("apple", 5)
        self.inv.remove_product("apple", 5)
        self.assertEqual(self.inv.get_product_quantity("apple"), 0)

    def test_remove_nonexistent_raises_error(self):
        with self.assertRaises(ValueError):
            self.inv.remove_product("banana", 1)

    def test_remove_excess_raises_error(self):
        self.inv.add_product("apple", 3)
        with self.assertRaises(ValueError):
            self.inv.remove_product("apple", 10)

    # --- get_product_quantity ---
    def test_get_quantity_nonexistent_returns_zero(self):
        self.assertEqual(self.inv.get_product_quantity("ghost"), 0)

    # --- total_inventory_value ---
    def test_total_inventory_value_correct(self):
        self.inv.add_product("apple", 10)
        self.inv.add_product("banana", 5)
        price_list = {"apple": 1.50, "banana": 0.80}
        self.assertAlmostEqual(
            self.inv.total_inventory_value(price_list), 19.0)
```

```
def test_total_value_missing_price_treated_as_zero(self):
    self.inv.add_product("mystery", 10)
    self.assertEqual(self.inv.total_inventory_value({}), 0)

def test_total_value_empty_inventory(self):
    self.assertEqual(self.inv.total_inventory_value({"apple": 2.0}), 0)

if __name__ == "__main__":
    unittest.main()
```

Verification & Validation (V & V)

V & V – Goal of each step (2024 Q2, 2025 Q13)

Verification: “Are we building the product *right*?”

Checks that the software correctly implements the specification – internal consistency, no implementation errors. Done by developers via reviews, inspections, walkthroughs, static analysis.

Validation: “Are we building the *right* product?”

Checks that the software meets the actual user needs and intended use. Done by running tests with real users/stakeholders. Includes acceptance testing.

Memory trick: Verification = spec check; Validation = user needs check.

Quality Management – Deming’s Cycle

Deming’s Cycle (PDCA) – 2025 Q12

The **Plan-Do-Check-Act** (PDCA) cycle is a continuous quality improvement framework:

1. **Plan:** Identify a problem or improvement opportunity; set goals; design a solution/experiment.
2. **Do:** Implement the plan on a small scale; collect data.
3. **Check (Study):** Analyse results against the goal; did it work?
4. **Act:** If successful, standardise and scale the change; if not, revise the plan and repeat.

In software context: Used in process improvement – identify quality issues, try a fix, measure impact, standardise if effective. Related to ISO/IEC standards and CMMI.

Concurrent Development & Git

How is Git useful for concurrent development? (2025 Q11)

Git is a **distributed version control system** that enables concurrent development through:

1. **Branching:** Each developer works on their own branch, isolating changes. Feature branches prevent conflicts in the main codebase until a feature is complete.
2. **Merging:** Git merges independently developed branches, automatically reconciling most changes and flagging conflicts for manual resolution.
3. **Conflict Detection:** When two developers edit the same lines, Git detects the conflict explicitly rather than silently overwriting, ensuring neither developer’s work is lost.
4. **History & Auditability:** Full commit history means any change can be traced, reverted, or compared – essential for debugging integration issues in large teams.

Project Tracking – Megathon Example

Megathon (Hackathon) – Initiatives, Activity List, Estimates, Priorities (2024 Q7, 2025 Q16)

Format: Kanban-style task board (can also be a table)

Initiative	Activity	Description	Effort	Priority
Pre-Event Setup	Venue booking	Book venue, arrange AV, internet	4h	High
Pre-Event Setup	Team registration	Build registration form/portal	6h	High
Pre-Event Setup	Sponsor outreach	Contact sponsors, confirm prizes	8h	High
Pre-Event Setup	Problem statements	Draft & review problem sets	6h	High
Event Execution	Check-in system	Participant check-in, ID verify	2h	High
Event Execution	Mentorship schedule	Assign mentors to teams	2h	Medium
Event Execution	Judging criteria	Define rubric for evaluation	3h	High
Post-Event	Demo presentations	Organise final pitch schedule	2h	Medium
Post-Event	Judging & awards	Score submissions, announce winners	3h	High
Post-Event	Feedback collection	Send survey to participants	1h	Low

Progress tracking: Use a Kanban board with columns: **Backlog** | **In Progress** | **Done**. Each activity is a card; move it as work progresses. High-priority items should be in “In Progress” first.

Quick-Reference Answer Sheet

Multiple Choice – Definitive Answers

Question	Answer	Year(s)
Use-case actors always people?	False – actors can be systems, devices, timers	2024
Timetable for start/finish of activities	Schedule	2024, 2025
Customer’s needs document	Request for Proposal (RFP)	2024, 2025
CPM stands for	Critical Path Method	2024
CPM primary benefit	Identifies longest task sequence / project duration	2025
Requirements Engineering activities	Elicitation, Modeling, Domain Analysis, Specification	2024, 2025
Acceptance tests conducted by	End Users (UAT)	2024
Risk-driven process model	Spiral	2024
TDD primary purpose	Write automated tests before code	2025
Best Agile tracking tool	Kanban board	2025
Best use-case elicitation approach	Interviews with key stakeholders	2025
Best requirements elicitation technique	JAD (Joint Application Development) sessions	2025
Multiplicities in UML	Specify number of instances per relationship	2025
Common unit-test assertion	assertTrue()	2025

One-Liners to Memorise

- **Spiral** = **risk-driven**; Waterfall = sequential; Agile = iterative; Prototype = throwaway model.
- **V&V**: Verification = right product built? | Validation = right product for the user?
- **Product Backlog** = prioritised list of all features owned by the Product Owner.
- **TDD** = Red → Green → Refactor (tests before code).
- **Regression testing** = re-run old tests after changes to ensure nothing broke.
- **Acceptance testing** = done by *end users* (UAT).

- **RFP** = customer document requesting proposals from vendors.
- **Requirements**: must be Correct, Unambiguous, Complete, Consistent, Verifiable, Traceable.
- **Deming's Cycle**: Plan → Do → Check → Act (continuous quality improvement).
- **Critical Path** = tasks with Slack = 0; determines minimum project duration.
- **JAD** = Joint Application Development = collaborative requirements workshops.
- **Kanban** = best Agile tracking tool (visualises flow, limits WIP).
- **Composition** (filled diamond) = strong aggregation; child cannot exist without parent.
- **Generalization** = inheritance; hollow triangle from subclass to superclass.

CPM Quick-Reference Formula Sheet

CPM Steps

1. Draw the network diagram (nodes = tasks, edges = dependencies).
2. **Forward Pass**: $ES(\text{start})=0$; $EF = ES + \text{Duration}$; ES of successor = $\max(EF \text{ of all predecessors})$.
3. **Backward Pass**: $LF(\text{end}) = EF(\text{end})$; $LS = LF - \text{Duration}$; LF of predecessor = $\min(LS \text{ of all successors})$.
4. **Slack** = $LS - ES = LF - EF$.
5. **Critical Path** = all tasks where Slack = 0.